Lab Assignment-4

School of Engineering and Technology
Course Name & Code: Fundamentals of Operating System Lab (ENCA252)
Program Name: BCA (AI & DS) (Research)

Name – Jyoti
Roll No – 2401201151
Course – BCA(AI & DS)
Section – A

Problem Title:

Implementation and Analysis of Disk Scheduling Algorithms

Problem Statement

Disk scheduling is an essential function of operating systems that determines the order in which disk I/O requests are serviced. Efficient disk scheduling improves system performance by reducing seek time and increasing throughput.

In this assignment, students will implement and analyze various disk scheduling algorithms such as **FCFS, SSTF, SCAN, and C-SCAN**. The goal is to simulate head movements and calculate total seek time.

Learning Objectives

- Understand disk scheduling concepts
- Implement various disk scheduling algorithms
- Calculate total head movement (seek time)
- Compare performance of different algorithms

Tools/Technology Used

- Python 3.x
- Basic Python libraries
- Any IDE (VS Code / PyCharm / IDLE)
- Linux/ Ubuntu

Assignment Tasks

Task 1: Input and Disk Request Representation

Objective:

To understand how disk requests are handled.

Description:

Students will input:

- Disk request queue
- Initial head position
- Disk size (optional) **Sub-Tasks:**

1. Input number of disk requests
2. Input request sequence
3. Input initial head position

Expected Outcome:

- Understanding of disk queue structure
- Ability to simulate disk head movement

```
(venv) bca_ai_ds_a@ubuntu:~/Downloads$ python ass4.py
Enter number of disk requests: 7
Enter request sequence: 82 170 43 140 24 16 190
Enter initial head position: 50
Enter disk size: 200

Request Queue: [82, 170, 43, 140, 24, 16, 190]
Initial Head Position: 50
```

Task 2: FCFS Disk Scheduling

Objective:

To implement First Come First Serve (FCFS).

Description:

Requests are serviced in the order they arrive.
Sub-Tasks:

1. Traverse requests sequentially

2. Calculate total seek time
3. Display movement sequence

Expected Outcome:

- Understanding simple scheduling
- Calculation of seek time

```
Request Queue: [82, 170, 43, 140, 24, 16, 190]
Initial Head Position: 50

--- FCFS ---
Move from 50 to 82 (Seek = 32)
Move from 82 to 170 (Seek = 88)
Move from 170 to 43 (Seek = 127)
Move from 43 to 140 (Seek = 97)
Move from 140 to 24 (Seek = 116)
Move from 24 to 16 (Seek = 8)
Move from 16 to 190 (Seek = 174)
Sequence: [82, 170, 43, 140, 24, 16, 190]
Total Seek Time: 642
```

Task 3: SSTF (Shortest Seek Time First)

Objective:

To implement SSTF algorithm.

Description:

Selects the request closest to current head position.

Sub-Tasks:

1. Find nearest request
2. Update head position
3. Calculate seek time

Expected Outcome:

- Understanding optimization strategy
- Reduced seek time compared to FCFS

```
--- SSTF ---
Move from 50 to 43 (Seek = 7)
Move from 43 to 24 (Seek = 19)
Move from 24 to 16 (Seek = 8)
Move from 16 to 82 (Seek = 66)
Move from 82 to 140 (Seek = 58)
Move from 140 to 170 (Seek = 30)
Move from 170 to 190 (Seek = 20)
Sequence: [43, 24, 16, 82, 140, 170, 190]
Total Seek Time: 208
```

Sample Python Logic (SSTF)

```
def sstf(requests, head):
seek_time = 0
 requests = requests.copy()
 while requests: nearest = min(requests,
key=lambda x: abs(x - head)) seek_time += abs(head - nearest)
head = nearest
requests.remove(nearest)

print("Total Seek Time:", seek_time)
```

Task 4: SCAN (Elevator Algorithm)

Objective:

To implement SCAN algorithm.

Description:

Disk head moves in one direction servicing requests, then reverses direction.

Sub-Tasks:

1. Sort request queue
2. Move in one direction
3. Reverse direction at end

Expected Outcome:

- Understanding elevator movement
- Balanced performance

```
--- SCAN ---
Move from 50 to 82 (Seek = 32)
Move from 82 to 140 (Seek = 58)
Move from 140 to 170 (Seek = 30)
Move from 170 to 190 (Seek = 20)
Move from 190 to 199 (Seek = 9)
Move from 199 to 43 (Seek = 156)
Move from 43 to 24 (Seek = 19)
Move from 24 to 16 (Seek = 8)
Sequence: [82, 140, 170, 190, 43, 24, 16]
Total Seek Time: 332
```

Sample Python Logic (SCAN)

```
def scan(requests, head, disk_size):
seek_time = 0
 left = [r for r in requests if r < head]
right = [r for r in requests if r >= head]

 left.sort(reverse=True)
right.sort()
 for r in right:
seek_time += abs(head - r)
head = r

 seek_time += abs(head - (disk_size - 1))
head = disk_size - 1
 for r in left:
seek_time += abs(head - r)
head = r

print("Total Seek Time:", seek_time)
```

Task 5: C-SCAN Algorithm

Objective:

To implement Circular SCAN.

Description:

Head moves in one direction only and jumps back to the beginning.

Sub-Tasks:

1. Move in one direction
2. Jump to start
3. Continue servicing

Expected Outcome:

- Uniform wait time
- Improved fairness

```
--- C-SCAN ---
Move from 50 to 82 (Seek = 32)
Move from 82 to 140 (Seek = 58)
Move from 140 to 170 (Seek = 30)
Move from 170 to 190 (Seek = 20)
Move from 190 to 199 (Seek = 9)
Jump from 199 to 0
Move from 0 to 16 (Seek = 16)
Move from 16 to 24 (Seek = 8)
Move from 24 to 43 (Seek = 19)
Sequence: [82, 140, 170, 190, 16, 24, 43]
Total Seek Time: 391
```

Sample Python Logic (C-SCAN)

```
def cscan(requests, head, disk_size):
seek_time = 0
 left = [r for r in requests if r < head]
right = [r for r in requests if r >= head]
 left.sort()
right.sort()
for r in right:
seek_time +=
abs(head - r)
head = r

 seek_time += abs(head - (disk_size - 1))
head = 0
 seek_time += disk_size - 1
 for r in left:
seek_time += abs(head - r)
head = r

print("Total Seek Time:", seek_time)
```

Task 6: Performance Comparison

Objective:

To compare all algorithms.

Description:

Students will compare:

- Total seek time

• Efficiency **Sub-Tasks:**

1. Run all algorithms
2. Compare outputs
3. Identify best algorithm

Expected Outcome:

• Analytical understanding
• Performance comparison

```
==================================
PERFORMANCE COMPARISON
==================================
FCFS Seek Time: 642
SSTF Seek Time: 208
SCAN Seek Time: 332
C-SCAN Seek Time: 391
```

Task 7: Result Analysis

Objective:
To interpret results.

Description:
Students will:

• Analyze which algorithm performs best
• Justify results **Sub-Tasks:**

1. Compare seek times
2. Identify pros/cons
3. Write conclusion **Expected Outcome:**

• Concept clarity
• Real-world understanding

```
==================================
ANALYSIS
==================================
Best Algorithm: SSTF (Minimum Seek Time)
Worst Algorithm: FCFS (Maximum Seek Time)
```